

LP5 – Complete Code Run Guide



Assignment 1 Folder – Java RMI (Arithmetic Operations)

Files: `ServerIntf.java`, `ServerImpl.java`, `Server.java`, `Client.java`

Prerequisites: Java JDK installed, `rmiregistry` available in PATH.

Steps (open 3 terminals in `d:\LLM-wiki\code\Assignment 1\`)

Terminal 1 – Compile all files

```
cd "d:\LLM-wiki\code\Assignment 1"
javac *.java
```

Terminal 2 – Start RMI Registry

```
cd "d:\LLM-wiki\code\Assignment 1"
start rmiregistry
```

Keep this terminal running. Wait 2-3 seconds before the next step.

Terminal 3 – Start the Server

```
java Server
```

Expected output:

```
server started !
```

Terminal 4 – Run the Client

```
java Client
```

Expected interaction:

```
Enter num1: 10
Enter num2: 5
Add is 15
sub is 5
mul is 50
div is 2
square is 100
squar root is 2
```

Assignment 2 Folder – Java CORBA (String Reversal)

Files: `ReverseModule.idl`, `ReverseServer.java`, `ReverseClient.java`,
`Reverseimpl.java`

Prerequisites: Java JDK ($\leq$ Java 8 recommended – CORBA was removed in Java 11+). Use JDK 8.

Steps (open terminals in `d:\LLM-wiki\code\Assignment 2\`)

Step 1 – Generate stubs from IDL

```
cd "d:\LLM-wiki\code\Assignment 2"
idlj -fall ReverseModule.idl
```

This creates a `ReverseModule/` folder with generated stubs/skeletons.

Step 2 – Compile all Java files

```
javac *.java ReverseModule/*.java
```

Terminal 2 – Start ORBD (Object Request Broker Daemon)

```
start orbd -ORBInitialPort 1050
```

Wait 2-3 seconds for ORBD to fully start.

Terminal 3 – Start the Server

```
java ReverseServer -ORBInitialPort 1050 -ORBInitialHost localhost
```

Expected output:

```
Server ready and waiting...
```

Terminal 4 – Run the Client

```
java ReverseClient -ORBInitialPort 1050 -ORBInitialHost localhost
```

Expected interaction:

```
Enter string to reverse: hello
```

```
Reversed string: olleh
```

[!WARNING]

CORBA(`orbd` , `idlj`) is **removed from JDK 11+**. If you get errors, switch to **JDK 8**.

Check your Java version: `java -version`

Assignment 3 Folder – MPI in C (Linux/Ubuntu Only)

Files: `Assignment3.c`

Prerequisites: Linux (Ubuntu) with OpenMPI installed. *(This will NOT work on Windows directly – use WSL or Ubuntu VM).*

Steps (run on Ubuntu/WSL terminal)

Step 1 – Install MPI (if not already installed)

```
sudo apt-get update
sudo apt install mpich
```

OR install OpenMPI manually (as per guide.txt):

```
# Download from http://www.open-mpi.org
tar -jxf openmpi-4.1.4.tar.bz2
cd openmpi-4.1.4
./configure --prefix=$HOME/opt/openmpi
make all
make install
```

```
# Add to PATH
```

```
echo "export PATH=\$PATH:\$HOME/opt/openmpi/bin" >> $HOME/.bashrc
echo "export LD_LIBRARY_PATH=\$LD_LIBRARY_PATH:\$HOME/opt/openmpi/lib" >>
source ~/.bashrc
```

Step 2 – Compile the C file

```
mpicc Assignment3.c -o mpi_program
```

Step 3 – Run with 4 processes

```
mpirun -np 4 ./mpi_program
```

If you get "not enough slots" error:

```
mpirun --oversubscribe -np 4 ./mpi_program
```

Assignment 4 Folder – Berkeley Clock Sync (Python)

Files: `server.py`, `client.py`

Prerequisites: Python 3, `python-dateutil` library.

Install dependency:

```
pip install python-dateutil
```

Steps (open 2 terminals in `d:\LLM-wiki\code\Assignment 4\`)

Terminal 1 – Start the Clock Server (Master)

```
cd "d:\LLM-wiki\code\Assignment 4"
python server.py
```

Expected output:

```
Socket at master node created successfully
Clock server started...
Starting to make connections...
```

```
Starting synchronization parallelly...
New synchronization cycle started.
Number of clients to be synchronized: 0
```

Terminal 2 – Start a Client (Slave)

```
python client.py
```

Expected output:

```
Starting to receive time from server
Starting to receive synchronized time from server
Recent time sent successfully
Synchronized time at the client is: 2026-05-13 20:34:10.123456
```

[!TIP]

You can open **multiple client terminals** to simulate multiple slave nodes syncing to the master.

Assignment 5 Folder – Token Ring Algorithm (Java)

File: `TokenRing.java`

Steps (1 terminal in `d:\LLM-wiki\code\Assignment 5\`)

Step 1 – Compile

```
cd "d:\LLM-wiki\code\Assignment 5"
javac TokenRing.java
```

Step 2 – Run

```
java TokenRing
```

Expected interaction:

```
Enter the number of nodes you want in the ring: 5
Ring Formed is as below:
0 1 2 3 4 0 (loop back to Node 0)
Note: Initially Token is at Node 0
```

```
Enter Sender: 2
Enter Receiver: 4
Enter Data to Send: 99
```

```
Sender is 2 and Sending Data is 99
Token Passing: [Previous Node -> Next Node]
0 -> 1 -> 2
Note: Current Token is at Node 2
```

```
Data Passing: [Previous Node -> Next Node]
2 -> 3 -> 4
Receiver: 4, Received the data: 99
```

```
Do you want to send data again? If yes Enter 1, If no Enter 0:
```



Assignment 6 Folder – Bully & Ring Election Algorithms (Java)

Files: `BullyAlgoExample.java`, `RingAlgorithm.java`

Steps (terminal in `d:\LLM-wiki\code\Assignment 6\`)

Step 1 – Compile

```
cd "d:\LLM-wiki\code\Assignment 6"
javac BullyAlgoExample.java RingAlgorithm.java
```

Step 2a – Run Bully Algorithm

```
java BullyAlgoExample
```

Expected interaction:

```
Enter total number of processes: 4
Status for process 1: 1
Priority of process 1: 2
Status for process 2: 1
Priority of process 2: 4
Status for process 3: 0
```

Priority of process 3: 5
Status for process 4: 1
Priority of process 4: 3
Enter process which will initiate election: 1
Election message is sent from 1 to 2
Election message is sent from 2 to 4
After electing process the final coordinator is 2

[!NOTE]

Status values: 1 = process is alive, 0 = process is dead (crashed).

Higher priority number wins the election in the Bully algorithm.

Step 2b – Run Ring Algorithm

```
java RingAlgorithm
```

Assignment 7 Folder – SOAP Web Service (NetBeans + GlassFish)

Location: d:\LLM-wiki\code\Assignment 7\Arithmetic Operation\

This assignment requires **NetBeans IDE** with **GlassFish Server** configured.

Steps

Part A – Create and Deploy the Web Service (Server)

1. Open **NetBeans IDE**
2. **File** → **New Project** → **Java Web** → **Web Application**
 - Name: `Assgn_7` → Finish
3. Right-click the project → **New** → **Web Service**
 - Service Name: `mycalculator`
 - Package: `com.myservice`
4. Open `mycalculator.java` – delete the auto-generated sample method body at the bottom
5. Right-click `mycalculator` under **Web Services** → **Add Operation**
 - Add operations: `add`, `subtract`, `multiply`, `divide` (each takes `double a`, `double b`)
6. Right-click project → **Build**, then **Deploy**
7. Right-click `mycalculator` → **Test Web Service** → browser opens with WSDL tester

Part B – Create the Client

1. **File** → **New Project** → **Java Web** → **Web Application**
 - Name: `CalculatorWebService_Client` → Finish
2. Open `index.html` (or `index.jsp`):
 - Create an HTML form with inputs for two numbers and a Submit button
 - Set form action to `CalculatorServlet`
3. Right-click project → **New** → **Web Service Client**
 - Paste the WSDL URL from Part A (e.g.
`http://localhost:8080/Assgn_7/mycalculatorService?wsdl`)
4. Right-click project → **New** → **Servlet**
 - Name it `CalculatorServlet` (matches form action)
5. In `CalculatorServlet.java` :
 - Drag the service methods from the Web Service References panel
 - Get inputs: `double a =`
`Double.parseDouble(request.getParameter("value1"));`
 - Call the remote method and write result to response
6. Right-click client project → **Clean and Build**
7. Right-click `index.html` → **Run File** → browser opens, test the calculator

Quick Reference Summary

Assignment Folder	Run Command(s)
Assignment 1(RMI)	<code>javac *.java → start rmiregistry → java Server → java Client</code>
Assignment 2(CORBA)	<code>idlj -fall ReverseModule.idl → javac *.java ReverseModule/*.java → start orbd -ORBInitialPort 1050 → java ReverseServer ... → java ReverseClient ...</code>
Assignment 3(MPI)	<code>mpicc Assignment3.c → mpirun -np 4 ./a.out (Linux only)</code>
Assignment 4(Berkeley)	<code>pip install python-dateutil → python server.py → python client.py</code>
Assignment 5(Token Ring)	<code>javac TokenRing.java → java TokenRing</code>
Assignment 6 (Bully/Ring)	<code>javac BullyAlgoExample.java RingAlgorithm.java → java BullyAlgoExample</code>

Assignment Folder	Run Command(s)
Assignment 7 (Web Service)	Use NetBeans IDE with GlassFish server